CS 3011: Artificial Intelligence

Introduction

Instructor: Dr. Durgesh Singh

CSE Discipline, PDPM IIITDM, Jabalpur -482005

Specifying the Task Environment

- In designing an agent, the first step must be to specify the task environments.
- The task environment is specified using four parameters:
  - Performance
  - Environment
  - Actuators
  - Sensors
- These four parameters are called as PEAS (**P**erformance, **E**nvironment, **A**ctuator, **S**ensor).

PEAS

- Agent: an automated taxi driver

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

PEAS

- Agent: Spam filter
  - Performance measure: Minimizing false positives, Minimizing false negatives
  - Environment: A user's email account, email server
  - Actuators: Mark as spam, delete, etc.
  - Sensors: Incoming messages, other information about user's account

PEAS

- Agent: Medical Diagnosis System
  - Performance measure: Healthy patient, minimize costs
  - Environment: Patient, hospital, staff
  - Actuators: Screen display (questions, tests, diagnoses, treatments, referrals)
  - Sensors: Keyboard/ voice entry of symptoms, findings, patient's answers.

PEAS

- Agent: Part-picking robot
  - Performance measure: Percentage of parts in correct bins
  - Environment: Conveyor belt with parts, bins
  - Actuators: Jointed arm and hand
  - Sensors: Camera, joint angle sensors.



PEAS

- Agent: Interactive English tutor
  - Performance measure: Maximize student's score on test
  - Environment: Set of students
  - Actuators: Screen display (exercises, suggestions, corrections)
  - Sensors: Keyboard.

Types of Environments

- An agent may have to act on the following types of environments.
  - Fully observable vs. Partially observable
  - Deterministic vs. Stochastic
  - Episodic vs. Sequential
  - Static vs. Dynamic
  - Discrete vs. Continuous
  - Single agent vs. Multiagent

Fully Observable vs. Partially Observable

- An environment is fully observable if an agent's sensors give it access to the complete state of the environment at each point in time.
- For fully observable environment, the sensors should detect all the aspects that are relevant to the choice of action (relevance depends upon the performance measure).
- In the fully observable environments, the agent need not to maintain internal state to keep the track of the world.
- Example : The environment of crossword puzzle, chess are fully observable.

Fully Observable vs. Partially Observable…

- An environment is partially observable if the entire state of the environment is not known to the agent due to noisy and inaccurate sensors or parts of the state are simply missing from the sensor data.
- The agent need to maintain internal state to keep the track of the world.
- For instance, a vacuum cleaner with local dirt sensor cannot tell whether dirt is in other squares or not.
- Similarly, an automated taxi cannot see what the other driver is thinking.
- If the agent has no sensors at all then the environment is unobservable.

Deterministic Vs. Stochastic

- An environment is deterministic, if the next state of the environment is completely determined by the current state and action executed by the agent.
- An agent need not worry about uncertainty in a fully observable, deterministic environment.
- Cross word puzzle, image analysis are examples of deterministic environments.
- An environment is stochastic (or nondeterministic), if the next state of the environment is not completely determined by the current state and action executed by the agent.

Deterministic Vs. Stochastic…

- Taxi driving is clearly stochastic in the sense, because one can never predict the behavior of the traffic exactly, some faults arise in the taxi without warning, etc.
- The vacuum world as we described it is deterministic, but variations can include nondeterministic elements such as randomly appearing dirt and an unreliable suction mechanism.
- If the environment is deterministic except for the action of the other agents, then the environment is strategic such as a chess or a poker playing agent.

Episodic vs. Sequential

- Episodic (vs. sequential): The agent's experience is divided into atomic "episodes"
  - Each episode consists of the agent perceiving and then performing a single action, and the choice of action in each episode depends only on the episode itself, e.g., a robot whose job is to detect faulty parts on a line in some factory. Part picking robot are also examples of episodic environments.
- In a sequential setting, the next episode depends on the previous one(s).
  - Also called a sequential decision process.

Episodic vs. Sequential…

- Chess and taxi driving are sequential; in both cases short term actions have long-term consequences.
- Episodic environments are much simpler than sequential environments because the agent does not need to think ahead.

Static vs. Dynamic

- If the environment can change while an agent is deliberating , then the environment is dynamic; otherwise, it is static.
- Static environments are easy to deal with because the agent need not to keep looking at the world.
- Crossword puzzle is an example of static environment.
- Dynamic environments, on the other hand, are continuously asking the agent what it wants to do.
- Taxi driving is clearly dynamic as the other cars are also moving while the driving algorithm decides what to do next.

Static vs. Dynamic…

- If the environment does not change with the passage of time but agent's performance score does, then the environment is semi-dynamic.
- Chess when played with a clock, is an example of semi-dynamic environments.

Discrete vs. Continuous

- The environment is discrete if the number of actions and possible states of the environment is finite otherwise it is continuous.
- A chess game has discrete environment as it has finite number of distinct states and a discrete set of percepts and actions.
- For instance, in case of taxi driving, the speed and location of taxi and other vehicles are continuously changing.
- The taxi driving actions are also continuous such as (steering, angles, etc.)

Single Agent vs. Multi Agent

- If only one agent is involved in the task, then the environment is single agent other wise it is multi agent.
- For instance, A person left alone in a maze is an example of the single-agent system whereas chess or taxi driving is a multi agent scenario.
- A multi agent environment can be competitive or cooperative.

Single Agent vs. Multi Agent...

- An agent is said to be in a competitive environment when it competes against another agent to optimize the output.
  - The game of chess is competitive as the agents compete with each other to win the game which is the output.
- An agent is said to be in a cooperative environment when multiple agents cooperate to produce the desired output.
  - When multiple self-driving cars are found on the roads, they cooperate with each other to avoid collisions and reach their destination which is the output desired.

Environment Types

	Chess with a clock	Chess without a clock	Taxi driving
Fully observable	Yes	Yes	No
Deterministic	Strategic	Strategic	No
Episodic	No	No	No
Static	Semi	Yes	No
Discrete	Yes	Yes	No
Single agent	No	No	No

- The environment type largely determines the agent design
- The real world is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent.

Agent types

- Four basic types in order of increasing generality:
  - Simple Reflex/ Reactive agents
  - Model-based Reflex / Deliberative agents
  - Goal-based agents
  - Utility-based agents
- And Finally: Learning agents

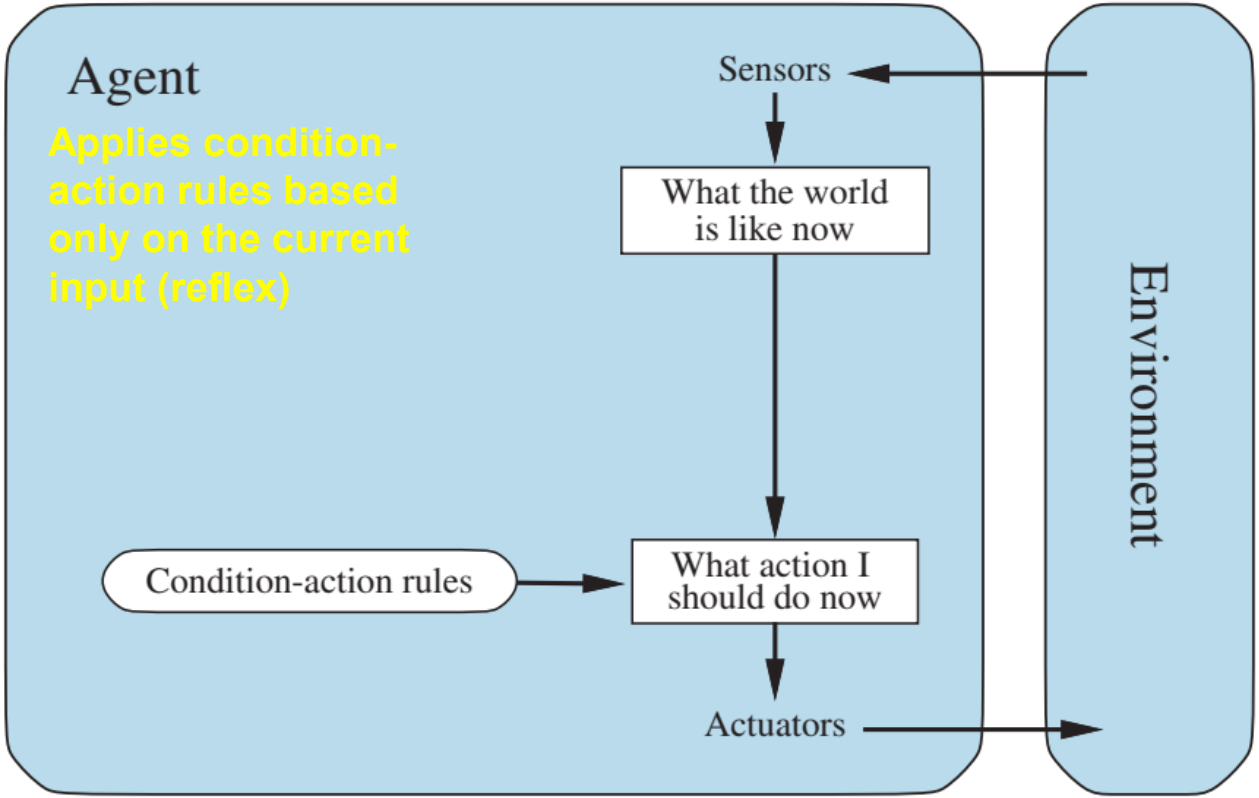
Simple Reflex Agents

- It is the simplest kind of agent.
- The agent selects the action based on the current percept only and ignoring the rest of the percept history.
- It uses just condition-action rules
  - The rules are like the form “if … then …”
- For instance, the simple-reflex program for a vacuum cleaner world is:

function REFLEX-VACUUM-AGENT([*location*,*status*]) **returns** an action

if *status* = *Dirty* **then return** *Suck*
else if *location* = *A* **then return** *Right*
else if *location* = *B* **then return** *Left*

Simple Reflex Agents …



Simple-Reflex Agents…

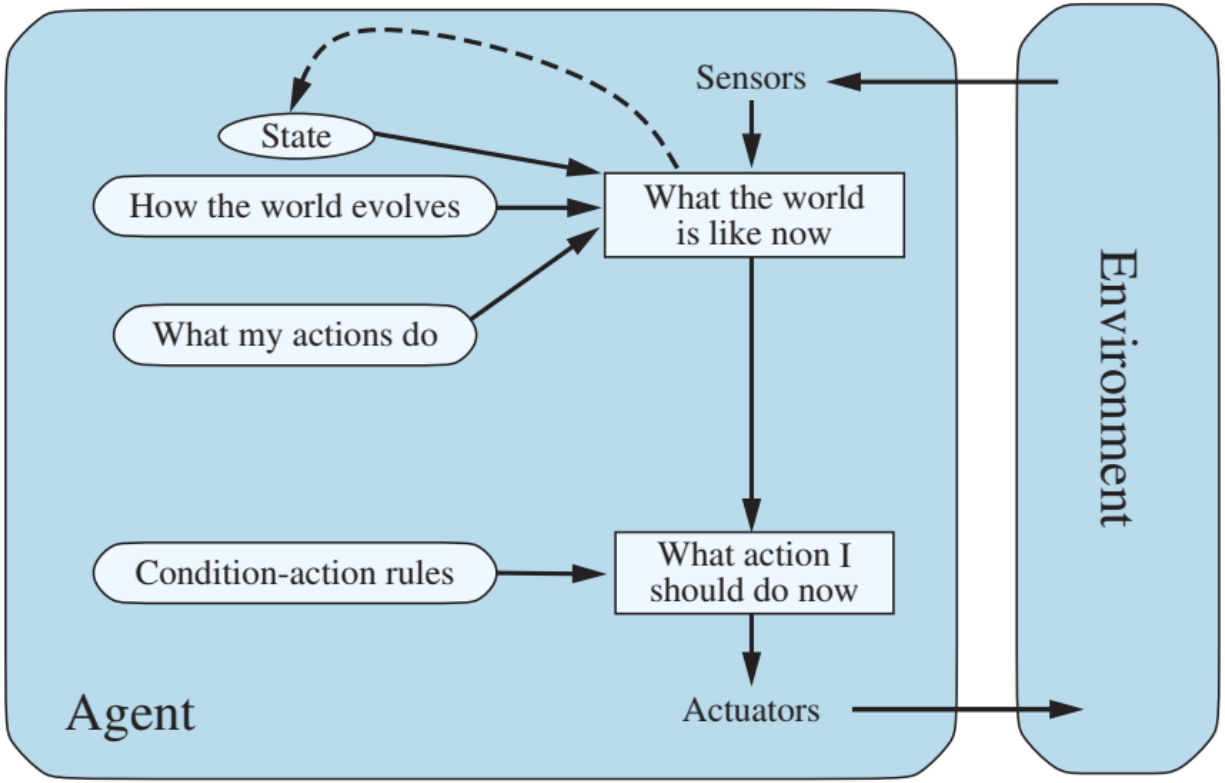
- It works only if the environment is **fully observable**.
- Simple reflex agents operating in partially observable environments, infinite loops are often unavoidable.
- Simple-reflex agents are simple, but they turn out to be of very limited intelligence.

Model-based Reflex agents

- Handle partial observability by keeping track of the part of the world it can't see now.
  - It does this by keeping an internal state that depends on what it has seen before so it holds information on the unobserved aspects of the current state. Ex. driving a car and changing lane.
- Agent keeps track of internal state that depends on the percept history.
- Updating the internal state information as time goes by requires two kinds of knowledge to be encoded in the agent program
  - Information about how the world evolves independently of the agent.
  - Information about how the agent's own actions affects the world.

Model-based Reflex agents

Handles Partial Observability by creating a model

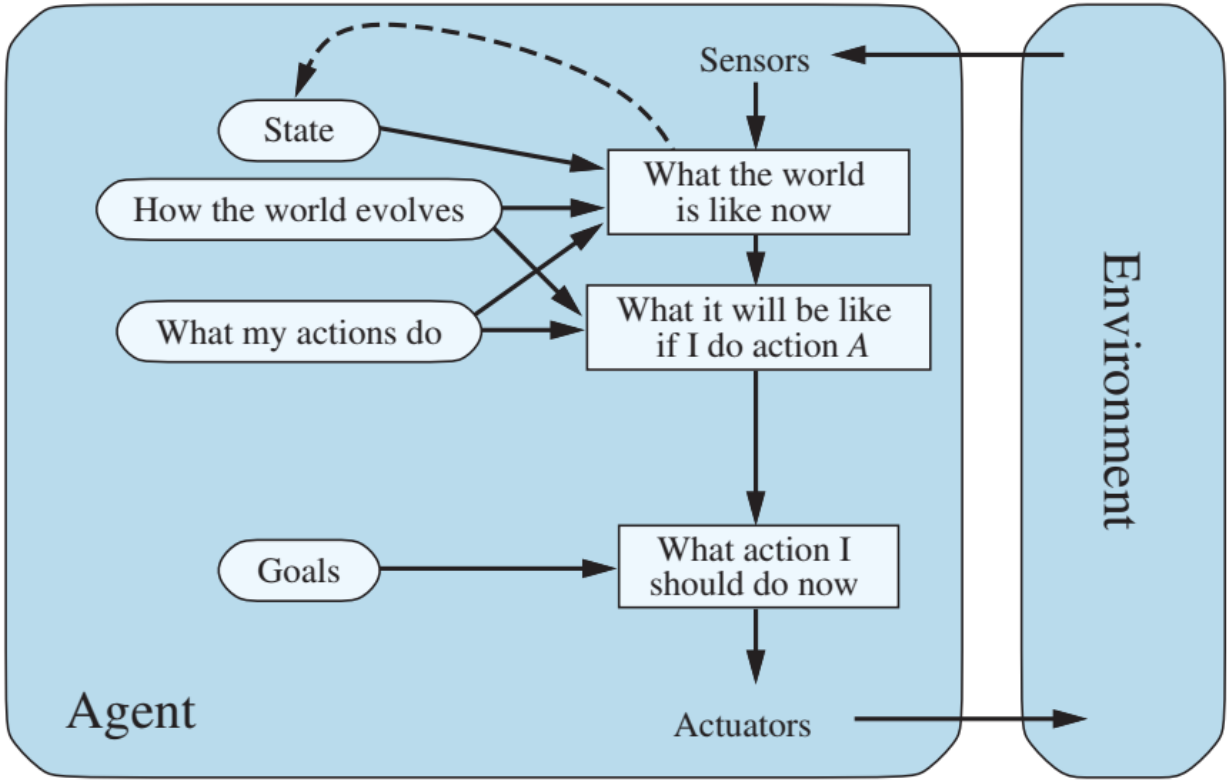


Goal-based agents

- The knowledge of the current state of the environment is not always sufficient.
- In addition to the current state, the agent needs some sort of goal information that describes situations that are desirable.
  - For example, at a road junction, the taxi can turn left, right or go straight on. The correct decision depends upon where the taxi is trying to go i.e., the passenger's destination.
- The agent program must combine the information about the possible actions with the goal information in order to achieve the goal.
- Consider the future with "What will happen if I do A?"

Goal-based agents

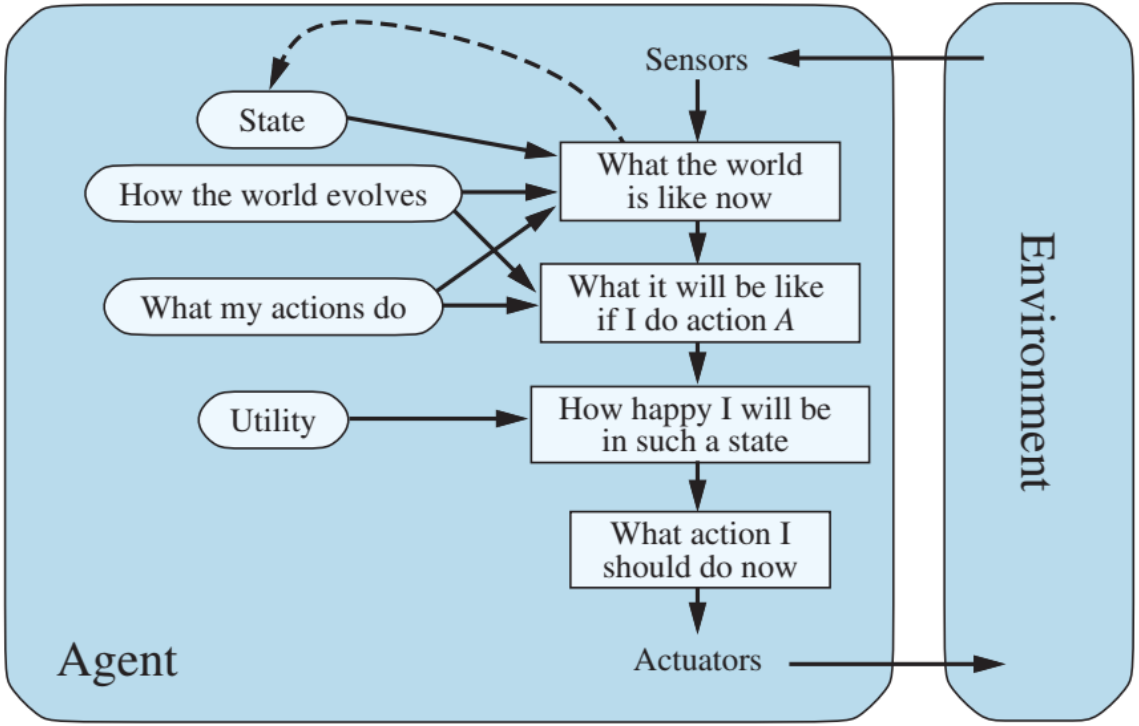
Along with the model, you need goals to direct the agent function.



Utility-based agents

- Sometimes achieving the desired goal is not enough. We may look for quicker, safer, cheaper trip to reach a destination.
- Agent happiness should be taken into consideration. We call it utility.
- A utility function maps a state onto a real number which describes the associated degree of happiness.
- A utility function is the agent's performance measure
- Because of the uncertainty in the world, a utility agent choses the action that maximizes the expected utility

Utility-based agents



Learning Agents

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:
  - Learning element
  - Critic
  - Performance element
  - Problem generator

Learning Agents

- **Learning element:** It is responsible for making improvements by learning from environment
- **Critic:** Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard.
- **Performance element:** It is responsible for selecting external actions. It is what we considered as agent so far.
- **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.

Learning Agents

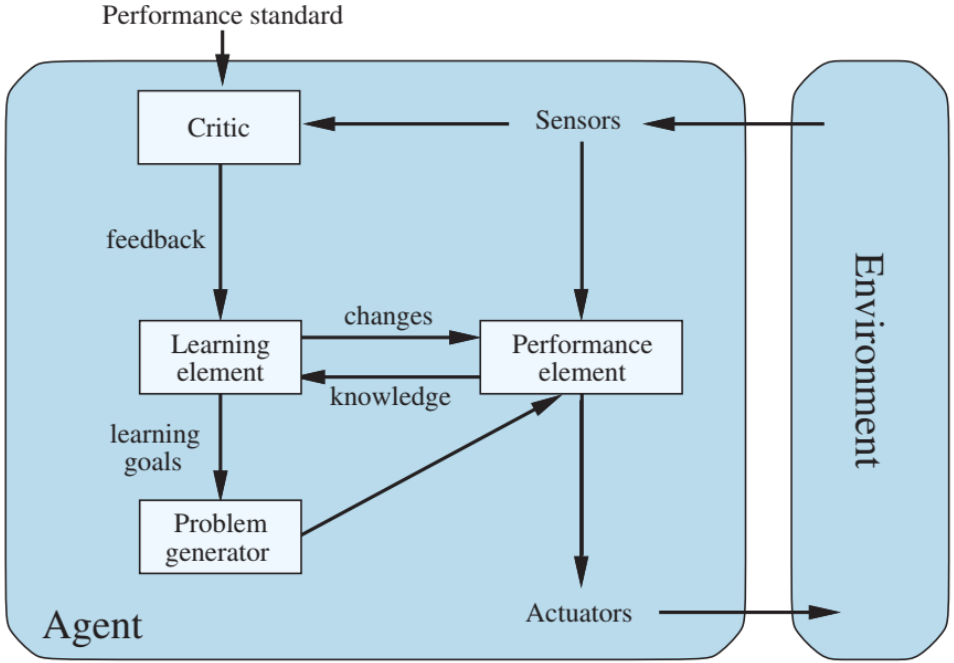


Fig: A general learning agent. The “performance element” box represents what we have previously considered to be the whole agent program. Now, the “learning element” box gets to modify that program to improve its performance.

Artificial Intelligence

SOLVING PROBLEMS BY SEARCHING

3 September 2026

Problem-Solving Agents

- This is a kind of goal-based agents that try to find a sequence of actions to achieve a goal.
  - Sometimes an action directly achieves a goal; sometimes a series of actions are required
- Finding this sequence of actions is called search

Example: Traveling in Romania

- Suppose the agent is currently in the Arad and has to reach Bucharest .
- Goal: Reach Bucharest on time

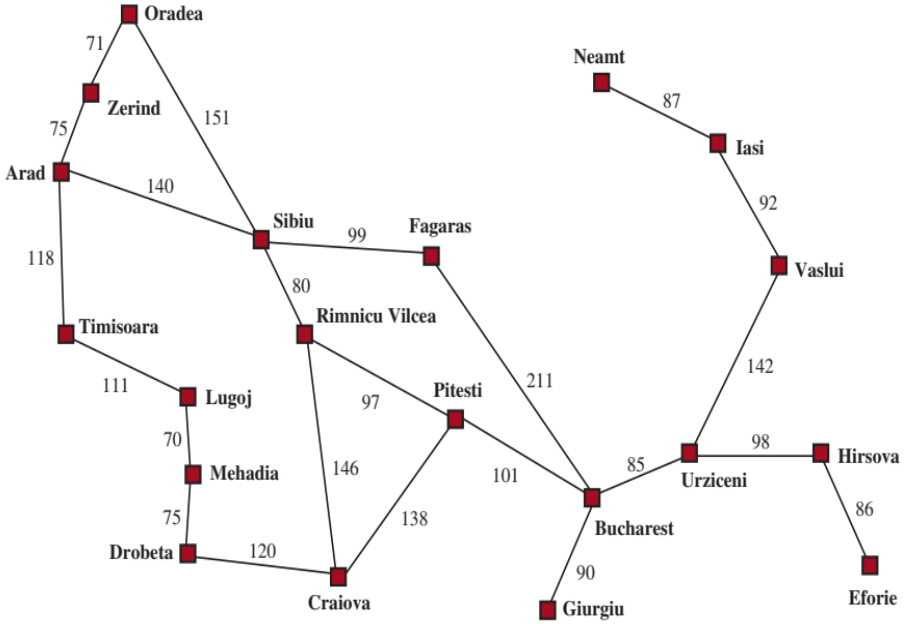


Figure: A simplified road map of part of Romania, with road distances in miles.

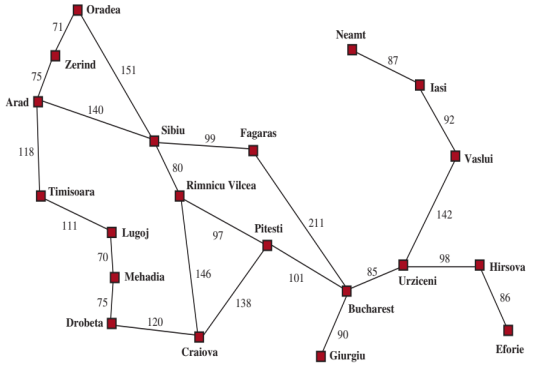
Traveling in Romania search Problem…

- Static: The configuration of the graph (the city map) is unlikely to change during search
- Observable: The agent knows the state (node) completely
- Discrete: Discrete number of cities and routes between them
- Deterministic: Transiting from one city (node) on one route, can lead to only one possible city
- Single-Agent: We assume only one agent searches at one time, but multiple agents can also be used.

Search problems and solutions

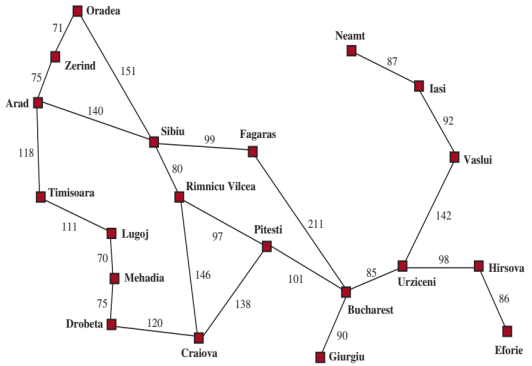
A search **problem** can be defined formally components:

- The initial state that the agent starts in. Example: *Arad*.
- A description of possible actions available to the agent. Given a state s, ACTIONS(s) returns a finite set of actions that can be executed in s.
  - Example: ACTIONS(Arad) = {ToSibiu, ToTimisoara, ToZerind}
- A transition model, which describes what each action does. RESULT(s,a) returns the state (successor) that results from doing action a in state s.
  - Example: RESULT(Arad, ToZerind) = Zerind



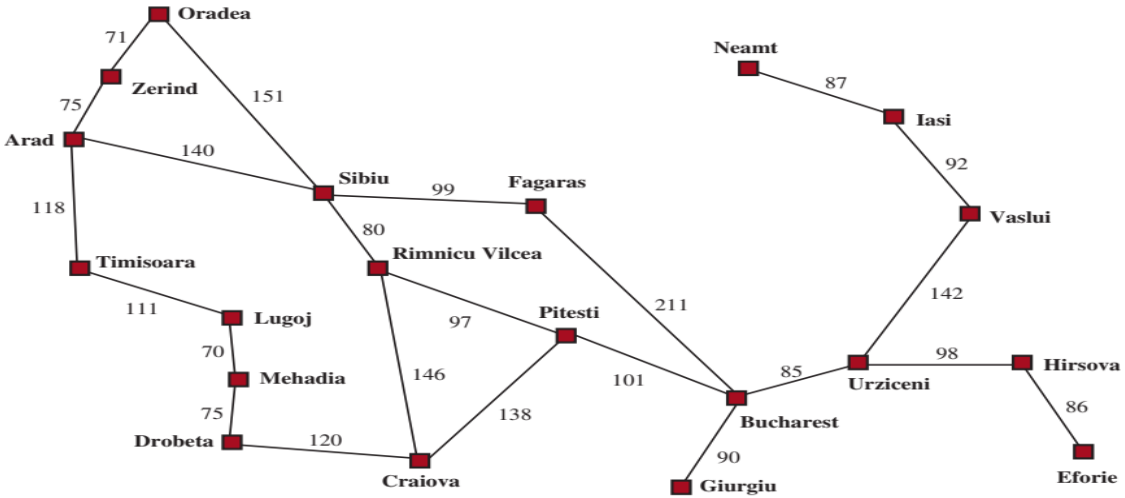
Search problems and solutions…

- state space: The set of all states reachable from the initial state by any sequence of action.
  - The initial state, actions, and transition model implicitly define the state space of the problem.
  - The state space forms a directed network (graph) in which the nodes are states, and the links are actions
- The goal test, which determines whether a given state is a goal state
  - Example:
- A path cost function assigns numeric cost to each path
  - A sequence of actions forms a path.



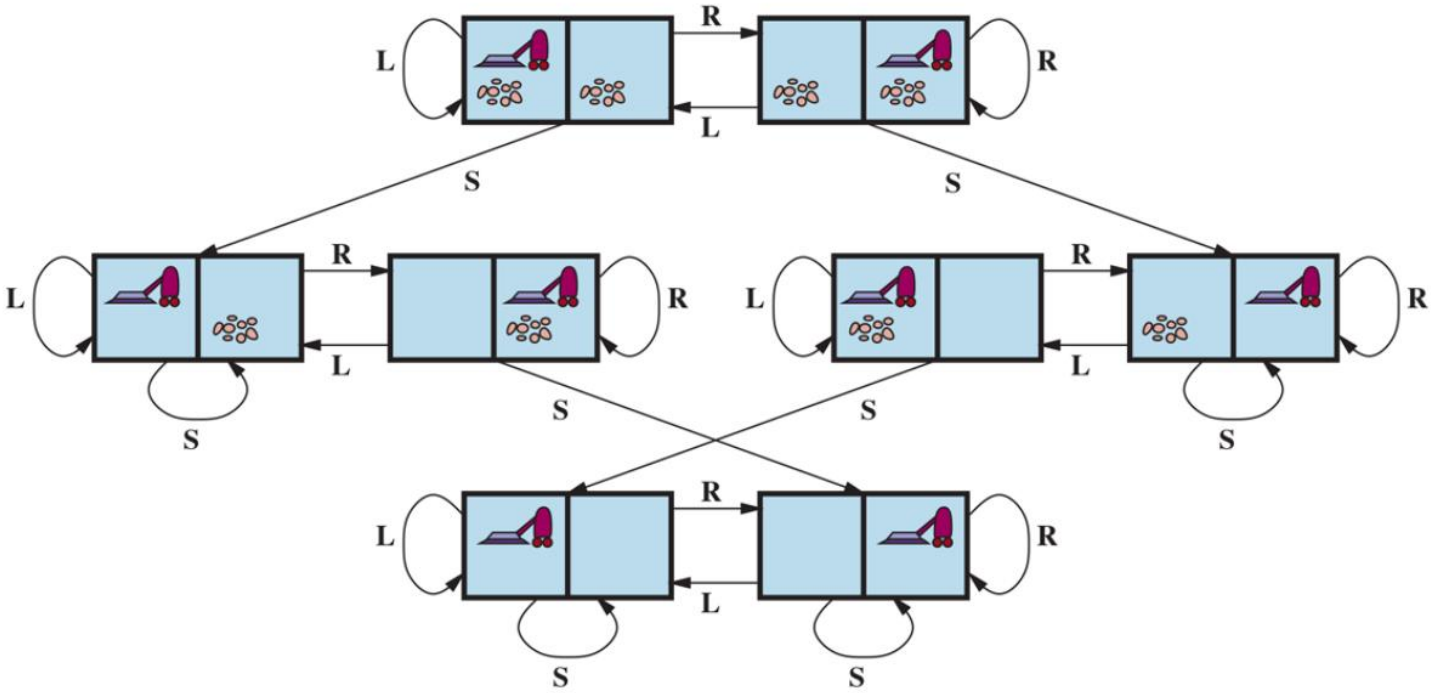
Optimal solution

- A solution to problem is a path from the initial state to a goal state.
  - Solution quality is measured by the path cost function.
- An optimal solution has the lowest path cost among all solutions.



Sample Search Problem

Vacuum World State Space Graph



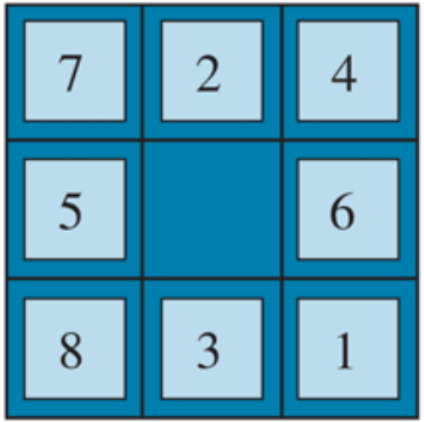
The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.

Formulating the Vacuum World Problem

- States: The state is determined by both the agent location and dirt locations. The agent is in one of the two locations, each of which might or might not contain dirt.
  - Possible world states= 2*2^2= 8
- Initial state: Any state may be designated as initial state
- Actions: Left, Right, and Suck
- Transition model: The actions have expected effects, except that moving Left in leftmost square, moving Right in rightmost square, and Sucking in a clean square have no effect.
- Goal State: Check whether all the squares are clean.
- Path Cost: Each cost 1, so the path cost is the number of steps in the path

Example: The 8-puzzle

- States?
- Actions?
- Goal test?
- Path cost?

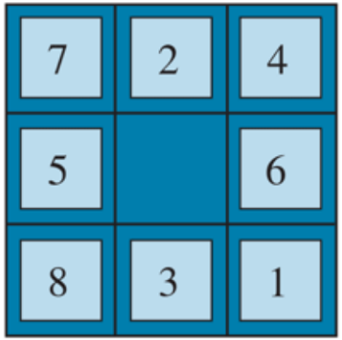


Start State



Goal State

Example: The 8-puzzle



Start State



Goal State

- States? locations of tiles
- Actions? move blank left, right, up, down
- Goal test? = goal state (given)
- Path cost? 1 per move

[Note: optimal solution of *n*-Puzzle family is NP-hard]

What exactly is “ Searching for Solution”?

- A solution is an action sequence
  - Search algorithms work by considering various action sequences.
- The possible action sequences, starting at the initial state form a search tree with
  - The initial state at the root
  - The branches are the actions and
  - The nodes correspond to states in the state space of the problem
- The essences for search
  - Following up one option now and putting others aside for later, in case the first choice does not lead to a solution

Infrastructure for search algorithms

- Search algorithms require a data structure to keep track of the search tree that is being constructed.
- Foe each node n of the tree, we have a structure with four components:

n.STATE: the state in the state space to which the node corresponds.

n.PARENT: the node in the search tree that generated this node.

n.ACTION: the action that was applied to the parent to generate the node.

n.PATH-COST: the total cost of the path (i.e., g(n)) from the initial state to this node.

Implementation: states vs. nodes

- A state is a (representation of) a physical configuration.
- A node is a data structure constituting part of a search tree includes state, parent node, action, path cost g(n).

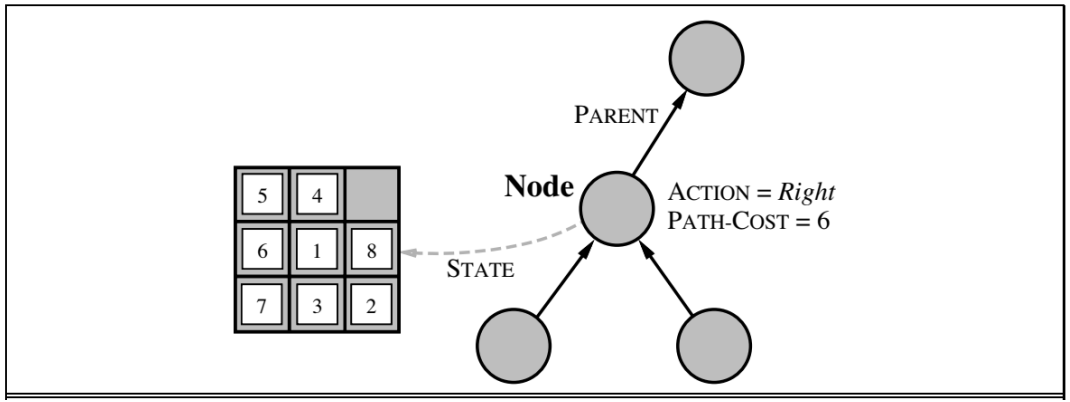
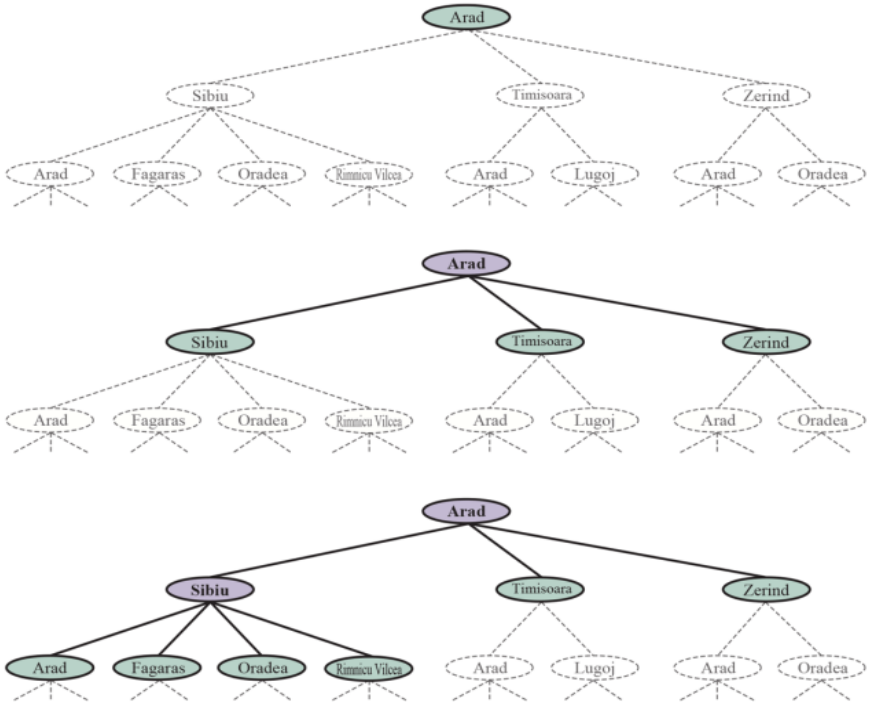


Fig: Nodes are the data structures from which the search tree is constructed. Each has a parent, a state, and various bookkeeping fields. Arrows point from child to parent.

A general Tree search algorithm

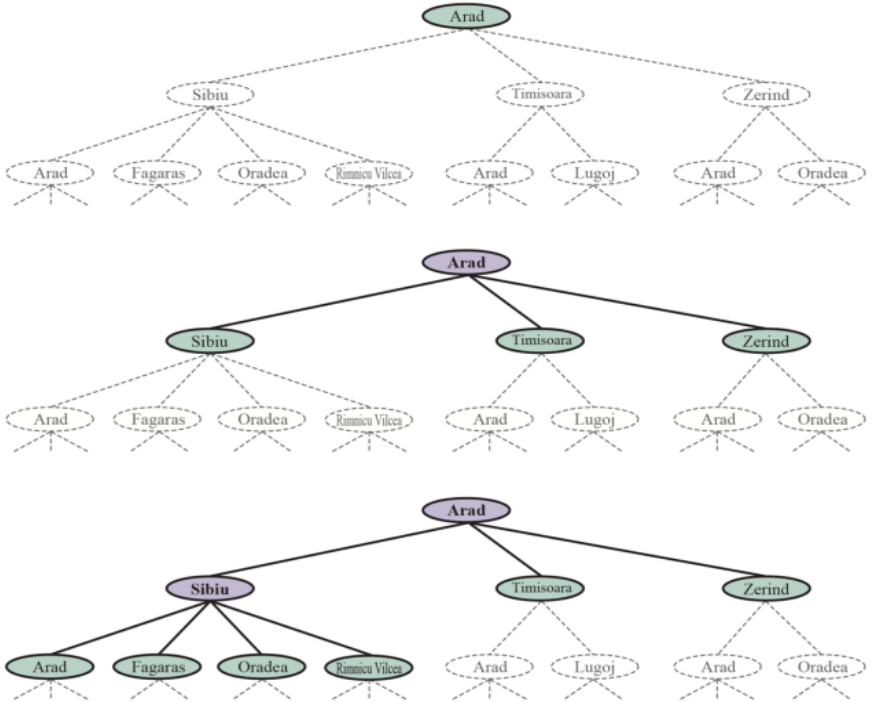
function TREE-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier



Three partial search trees for finding a route from Arad to Bucharest. Nodes that have been *expanded* are lavender with bold letters; nodes on the frontier that have been *generated* but not yet expanded are in green; the set of states corresponding to these two types of nodes are said to have been *reached*. Nodes that could be generated next are shown in faint dashed lines. Notice in the bottom tree there is a cycle from Arad to Sibiu to Arad; that can't be an optimal path, so search should not continue from there.

A general Graph search algorithm

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  initialize the explored set to be empty
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set



Three partial search trees for finding a route from Arad to Bucharest. Nodes that have been *expanded* are lavender with bold letters; nodes on the frontier that have been *generated* but not yet expanded are in green; the set of states corresponding to these two types of nodes are said to have been *reached*. Nodes that could be generated next are shown in faint dashed lines. Notice in the bottom tree there is a cycle from Arad to Sibiu to Arad; that can't be an optimal path, so search should not continue from there.

Search Strategies

- A search strategy is defined by picking the order of node expansion
- Strategies are evaluated along the following dimensions:
  - Completeness: Does it always find a solution if one exists?
  - Time complexity: Number of nodes generated
  - Space complexity: Maximum number of nodes in memory
  - Optimality: Does it always find a least-cost solution?
- Time and space complexity are measured in terms of
  - *b:* maximum no. of successors of any node
  - *d:* depth of the shallowest goal node
  - *m*: maximum length of any path in the state space.

Artificial Intelligence

Uninformed Search

3 September 2026

Uninformed Search Strategies

- It is also called **blind search**
- Uninformed search strategies use only the information available in the problem definition
- All they can do is generate successors and distinguish a goal state from a nongoal state.
- Example
  - Breadth-first search
  - Uniform-cost search
  - Depth-first search
  - Depth-limited search
  - Iterative deepening search

Breadth-first search

- Breadth-first search is a simple strategy in which the root node is expanded first
  - then all the successors of the root node are expanded next, then their successors, and so on
- In general, all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.
  - Expand shallowest unexpanded node
- Implementation:
  - This is achieved very simply by using a FIFO queue for the frontier.

Breadth-first search

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
      if EMPTY?(frontier) then return failure
      node ← POP(frontier)   /* chooses the shallowest node in frontier */
      add node.STATE to explored
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          if child.STATE is not in explored or frontier then do
              if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
              frontier ← INSERT(child, frontier)
```

